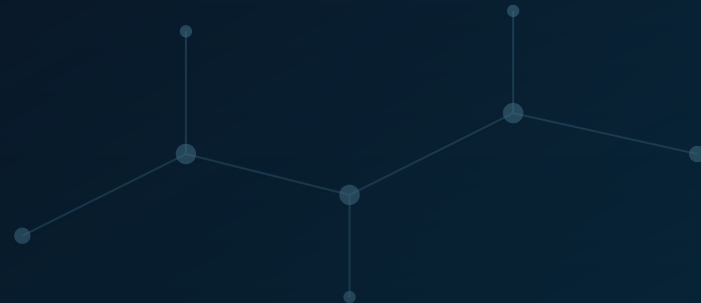


INF 345 — Fundamentals of DevOps

Lecture 5: Container Networking & Volumes

Adil Akhmetov · Lesson 5



Recap — Lesson 4 (Images, Layers & Multi-Stage Builds)

- Why does a multi-stage build produce a smaller final image? (only the last stage ships; earlier stages — compilers included — are thrown away except for what you `COPY --from=`)
- What does `podman history` show you? (every layer, its size, and the instruction that created it)
- Why copy `requirements.txt` and install it before copying the rest of the app? (keeps that layer's cache valid when only app code changes)

Today's agenda

- ☐ Why containers need networking, and network namespaces
- ☐ Publishing ports: `-p` , `EXPOSE` , and where to bind
- ☐ Default bridge vs user-defined networks, and DNS by name
- ☐ Podman specifics: rootless networking, pods
- ☐ Volumes: named, bind mounts, tmpfs — and where your data really lives
- ☐ Compose: wiring it all together
- ☐ Live demo → straight into today's practice

The scenario

Your week 4 app runs great as a single container. Now it needs a database. You start a second container, hardcode its IP in the app's config, and it works — until you restart the database container and get a different IP. Or you publish the database's port to the host "just to debug it" and forget to remove that flag.

Today is about the two things that make that scenario safe: how containers find each other, and where their data actually lives.

Block 1

Networking fundamentals

Isolation by default

- Every container gets its own **network namespace** by default — its own loopback, its own interfaces, its own routing table.
- Two containers can't see each other's ports or processes unless you explicitly connect them.
- Same namespace mechanism from lesson 3 (containers 101) — just applied to networking instead of the filesystem.

A container's `localhost` is its own — it is **not** the host's localhost, and it is not another container's localhost either (with one exception we'll get to: pods).

Publishing a port

```
podman run -d -p 8080:80 nginx
# host:container - host port 8080 forwards to container port 80
```

- Without `-p`, the container's port exists only inside its own network namespace — nothing on the host can reach it directly.
- `EXPOSE 80` in a Containerfile is **documentation** — it records intent, it does not publish anything by itself.
- `-p host:container` at run time is what actually punches the hole.
- (docker: identical flag — `docker run -p 8080:80 ...`)

Binding to 127.0.0.1 vs 0.0.0.0

```
podman run -d -p 127.0.0.1:8080:80 nginx # host-only
podman run -d -p 8080:80 nginx           # binds 0.0.0.0
```

- `-p 127.0.0.1:8080:80` binds only the host's loopback — reachable only from this machine.
- `-p 8080:80` (no address) binds `0.0.0.0` — reachable from any network this host is on.

On a shared or cloud host, `-p 8080:80` is reachable from the network unless a firewall stops it. Bind to `127.0.0.1:` for anything that should only be reached through a reverse proxy on the same machine.

Default bridge vs user-defined networks

	Default bridge	User-defined network
Created by	Automatically	You: <code>podman network create</code>
DNS by container name	No	Yes
Reach other containers by	IP address only	Container name
Isolation from other networks	One flat shared network	Only attached containers can talk

This is the single most common "my app can't reach my database" bug: two containers on the default bridge, app trying to connect to `db` by name — and it fails, because only user-defined networks get built-in DNS.

podman network create/ls/inspect

```
podman network create app-net
podman network ls
podman network inspect app-net
podman run -d --name db --network app-net postgres:16
podman run -d --name api --network app-net myapp
# from inside "api":
#   curl http://db:5432 → resolves "db" by name – DNS on app-net
```

(docker: identical commands — `docker network create` , `docker network ls` , `docker network inspect`)

Container-to-container communication

- On a shared user-defined network, containers reach each other by name over their **container** ports — no `-p` needed at all.
- `-p` is for the outside world (your browser, a client, a script on the host). It is not what lets two containers talk to each other.
- Rule: only publish the ports that something *outside* the container network actually needs to reach.

Never publish a database port. If your app and your database are on the same user-defined network, the app reaches the database by name on its container port — publishing 5432 to the host just gives every scanner on the internet a free shot at your data.

Quick check

Your app container can `curl http://db:5432` successfully with no `-p` flag anywhere. Why does that work?

Both containers are on the same user-defined network — its built-in DNS resolves `db` to the container's address, and container-to-container traffic doesn't need a host port published at all.

Block 2

Podman specifics: rootless networking & pods

Rootless networking: netavark & pasta

- Podman runs containers as your normal user by default — no root daemon. That changes how networking has to work under the hood.
- **Netavark** is Podman's network stack for container-to-container and bridge networking (the default since Podman 4).
- **Pasta** handles a rootless container's connection out to the host/internet, without needing root privileges or special capabilities.
- You won't configure these directly in this course — just recognize the names if you see them in `podman network inspect` or troubleshooting docs.

Pods: containers that share a network namespace

- A **pod** in Podman groups containers so they share **one** network namespace — the same idea as a Kubernetes pod, running locally.
- Containers in the same pod reach each other over `localhost` — no network, no DNS needed.
- Useful for a tightly-coupled sidecar next to your app — not a replacement for a user-defined network between independent services.

```
podman pod create --name mypod -p 8080:80
podman run -d --pod mypod --name web nginx
podman run -d --pod mypod --name sidecar myapp
# inside "sidecar": curl http://localhost:80 → reaches "web"
```

Block 3

Storage & volumes

The writable layer is ephemeral

- Every running container gets a thin **writable layer** on top of its image's read-only layers (lesson 4).
- Anything written there — logs, uploaded files, a SQLite file — disappears the moment `podman rm` removes that container. `stop` / `start` keeps it; `rm` does not.
- Fine for stateless apps. A data-loss bug waiting to happen for anything that needs to survive a restart or a redeploy.

If data needs to outlive the container, it can't live in the writable layer. That's what volumes are for.

Three ways to attach storage

	Named volume	Bind mount	tmpfs
Managed by	Podman/Docker	You (host path)	Podman/Docker, in memory
Survives container removal	Yes	Yes (it's a host directory)	No
Good for	App data, databases	Config, source during dev	Secrets, caches
Example	<code>-v mydata:/var/lib/db</code>	<code>-v ./src:/app/src:Z</code>	<code>--tmpfs /run</code>

Default choice for anything you'd call "the database's data": a named volume. Podman/Docker manages where it actually lives on disk — you don't need to know or care.

podman volume create/ls/inspect

```
podman volume create mydata
podman volume ls
podman volume inspect mydata
podman run -d --name db -v mydata:/var/lib/postgresql/data postgres:16
```

(docker: identical — `docker volume create` , `docker volume ls` , `docker volume inspect`)

A named volume outlives any single container. Remove and recreate the "db" container with the same `-v mydata:...` , and the data is still there.

:Z and :z — SELinux labels on bind mounts

- On SELinux-enforcing hosts (Fedora, RHEL — what DO188 runs on), a container can't read/write a bind-mounted host directory until it's relabeled for container access.
- :z — **shared** label: multiple containers can use this mount.
- :Z — **private** label: only this container can use it. Prefer :Z unless you specifically need to share the mount.

```
podman run -d -v ./src:/app/src:Z myapp
```

Forgetting :Z / :z on an SELinux host is the most common cause of a mysterious "permission denied" on a bind mount that a plain `ls` shows you clearly have access to.

Persistence across container removal

- A named volume or bind mount's data exists independently of the container using it — `podman rm` the container, the data stays.
- An **anonymous volume** (created implicitly, e.g. by an image's `VOLUME` instruction with no `-v` naming it) also survives `rm` by default — but nothing refers to it by name, so it quietly accumulates until you `podman volume prune` it.
- Bottom line: know whether your data is in a *named* volume before you tear a container down.

down vs down -v — the dangerous flag

- `podman-compose down` (or `docker compose down`) stops and removes containers and the default network — named volumes are left alone.
- `podman-compose down -v` additionally **deletes every named volume** the compose file defines. That's your database's data, gone.
- There is no confirmation prompt.

`-v` on `down` is the single most common way to accidentally delete a week's worth of test data. Read the flag before you type it.

Quick check

You stop and remove a container with `podman rm`. Its data was in a named volume. Is the data gone?

No — a named volume exists independently of any container. The data is still there; nothing currently references it until you start a new container with the same `-v`.

Block 4

Wiring it together with Compose

compose.yaml — services, networks, volumes

- `services:` — one entry per container: image, ports, environment, volumes — same options as `podman run`.
- `networks:` — user-defined networks Compose creates for you; every service on the same network gets DNS by name automatically.
- `volumes:` — named volumes declared once, referenced by any service.
- `depends_on:` — controls start order (not "wait until ready" — just "start after").
- `environment:` — env vars, same as `-e` on `podman run`.

`podman-compose` reads this same `compose.yaml` ; `docker compose` is the Docker equivalent. Both use the same file format.

Worked example — app + Redis

```
services:
  app:
    build: ./app
    ports:
      - "8080:8080"
    environment:
      - REDIS_HOST=redis
    depends_on:
      - redis
    networks:
      - app-net

  redis:
    image: redis:7-alpine
    volumes:
      - redis-data:/data
    networks:
      - app-net
    # no "ports:" here — redis is never published to the host

networks:
  app-net:
```

Why this shape matters

- `redis` isn't published — nothing outside this compose project can reach it directly. The "don't publish a database port" rule from Block 1, applied to Compose.
- `app` reaches `redis` at `redis:6379` — by name, over `app-net` — the same user-defined-network DNS from Block 1.
- `redis-data` is a named volume — `podman-compose down` (no `-v`) leaves the data intact across restarts.
- `depends_on` starts Redis's *container* before the app's — it does not wait for Redis to finish booting. A real app should still retry its connection on startup.

This exact shape — app + Redis, Redis internal-only, named volume — is today's practice.

Block 5

Debugging checklist

When it doesn't work

Symptom	Likely cause
Can't reach another container by name	Both on the default bridge, not a user-defined network
<code>permission denied</code> on a bind mount	Missing <code>:Z</code> / <code>:z</code> on SELinux, or a UID mismatch
Data gone after a rebuild	It was in the writable layer or an anonymous volume, not a named volume
Data gone after <code>down</code>	Someone ran <code>down -v</code>

Networking-shaped bug → check `podman network inspect <name>` for who's actually attached. Storage-shaped bug → check `podman volume inspect <name>` for where the data actually lives.

Block 6

Today's practice

Today's practice — wire app + Redis with Compose

Practice 05 gives you `app/` (a small counter API) and a starter `compose.yaml`. You fill in:

1. A user-defined network shared by both services
2. Redis on a named volume, **not** published to the host
3. `depends_on` so the app starts after Redis
4. An environment variable pointing the app at `redis` by name

```
podman-compose up -d
curl localhost:8080/count
curl localhost:8080/count
podman-compose down
podman-compose up -d
curl localhost:8080/count
# → count kept increasing — the restart didn't reset it
```

Graded automatically: Redis not published, named volume used, counter survives a `down` + `up -d` cycle (not `down -v` — that's a different result).

By the end of this lesson, you should be able to

- ☐ Explain why two containers can't reach each other by name on the default bridge network
- ☐ Publish a port correctly, and explain when to bind to `127.0.0.1` instead of `0.0.0.0`
- ☐ Choose a named volume, bind mount, or tmpfs for a given storage need
- ☐ Write a `compose.yaml` that keeps a database off the host network and its data in a named volume

Before next lecture

- ☐ Finish today's practice if you didn't wrap it up in session — same Maru submission flow as last time
- ☐ Keep the RHA DO188 lab moving — due next week (Week 6)

Week 6 is lab work only — this is your last lecture before that lab is due.

Next lecture

Container security & module wrap-up — locking down what we've built, and closing out the containers module.